

CS & AI Spaced **Repetition** Engine

A terminal-based learning tool for mastering complex computer science concepts.

STUDENT PROJECT • AFSHIN MALEK NEZHAD YAZDI

Project Overview

This project is a sophisticated C++ terminal application specifically designed to help master complex algorithms, data structures, and machine learning concepts through scientifically-proven spaced repetition.

It ingests external CSV datasets of technical questions, dynamically tests the user based on their historical accuracy (streak), and persistently saves their progress back to the local database.

Core Features: Data & Logic

Dynamic Loading

Automatically parses multiple external CSV datasets (Algorithms and Machine Learning) into dynamically allocated arrays of structs.

Spaced Repetition

Intelligently tracks user accuracy (streak). It optimizes study sessions by only presenting questions that require further study ($\text{streak} < 2$).

$O(\log n)$ Search

Allows users to instantly look up specific questions across multiple datasets using unique question IDs, ensuring rapid information retrieval.

Core Features: Storage & Analytics

Persistent Storage

Employs C++ File I/O to dynamically overwrite the source CSV files. This ensures that study progress and accuracy streaks are saved securely between application sessions without data loss.

Session Analytics

Calculates and displays real-time session statistics to keep the user informed of their progress. Metrics include total questions answered, correct/incorrect counts, and overall accuracy percentages.

Development Timeline

Week 1-2

Define core idea, set up GitHub repository, and establish project architecture.

Week 3-4

Build core program features, including CSV parsing and spaced repetition logic.

Week 5

Comprehensive testing, edge-case handling, and program debugging.

Week 6-7

Draft Output.pdf with screenshots and finalize GitHub submission.

Deliverables & Success Criteria

- **Program Files:** Complete C++ source code including main.cpp and supporting header/implementation files.
- **Documentation:** Output.pdf detailing program screenshots and comprehensive technical explanations.
- **Version Control:** A properly structured GitHub repository submission.
- **Success Criteria:** The project runs correctly without errors, includes all required features, and is submitted strictly on time.

System Architecture (main.cpp)

Initialization & Loading

The program instantiates two distinct module objects to handle different subject domains securely in memory.

- **Modularity:** AlgorithmQuiz and MachineLearningQuiz handle specific datasets independently.
- **Dynamic Ingestion:** Calls loadDataset() at runtime to parse CSV files cleanly into system memory.

```
main.cpp

int main() {
    AlgorithmQuiz algoModule(100);
    MachineLearningQuiz mlModule(100);

    // Store filenames for reuse
    string algoFile = "algorithms.csv"
    ;
    string mlFile = "ml_systems.csv";

    cout << "Loading datasets..." <<
        endl;
    algoModule.loadDataset(algoFile);
    mlModule.loadDataset(mlFile);
    cout << "System ready.\n" << endl;

    // ... enters menu loop ...
}
```

Handling Input & Persistence

Control Flow

```
if (choice == 1) {
    algoModule.studySession();
} else if (choice == 2) {
    mlModule.studySession();
} else if (choice == 3) {
    int searchID; cin >> searchID;
    algoModule.searchByID(searchID);
    mlModule.searchByID(searchID);
} else if (choice == 4) {
    // Save directly back to CSV!
    algoModule.saveToFile(algoFile);
    mlModule.saveToFile(mlFile);
}
```

Control Flow

A continuous do-while loop captures user intent and directs traffic to the appropriate module.

- **Study Routing:** Triggers specific studySession() logic.
- **Global Search:** Search requests span across both database modules natively.
- **Safe Exit:** saveToFile() guarantees streaks persist before exiting!

Data Persistence (File I/O)

Reading & Writing CSVs

The system dynamically stores the user's progress using the standard `<fstream>` library, avoiding the need for a heavy external database server.

- **Serialization:** Formats the struct array back into comma-separated values.
- **Streak Tracking:** Crucially saves the 5th column (streak) so users never lose their historical accuracy data between sessions.

- `StudyModule::saveToFile`

```
void StudyModule::saveToFile(string
    filename) {
    ofstream fileOut(filename);
    if (fileOut.is_open()) {
        fileOut << "ID,Topic,Prompt,
                    Answer,Streak\n";
        for (int i = 0; i < count; i++)
        {
            // Serialize user data &
            // streaks
            fileOut << deck[i].id << ", "
                    << deck[i].topic << ", "
                    << deck[i].prompt << "
                    << ", "
                    << deck[i].answer << "
                    << ", "
                    << deck[i].streak << "
                    << "\n";
        }
    }
}
```

Algorithm: Spaced Repetition

```
StudyModule::studySession

for (int i = 0; i < count; i++) {
    // Only test if streak is low
    if (deck[i].streak < 2) {
        cout << "Q: " << deck[i].prompt;

        // ... user input handling ...

        if (input == "y" || input == "Y")
        {
            deck[i].streak++; // Promote
                               streak!
            sessionCorrect++;
        } else {
            deck[i].streak = 0; // Reset
                               streak
            sessionIncorrect++;
        }
    }
}
```

The Core Logic

The engine isolates study material by continuously filtering the array based on performance metadata.

- **Targeted Learning:** Checking if $\text{streak} < 2$ guarantees the user only studies unmastered questions.
- **Dynamic Promotion:** Correct answers increment the streak.
- **Harsh Penalties:** Incorrect answers reset the streak to 0, forcing re-evaluation.

Information Retrieval

Binary Search Implementation

To support large datasets of questions, the engine employs a highly efficient $O(\log n)$ search algorithm instead of a linear scan.

- **Prerequisite Sorting:** `sortDeck()` runs during initialization (via Bubble Sort) to ensure data is ordered by ID.
- **Divide & Conquer:** The search space is halved every iteration, providing near-instant lookups.

StudyModule::searchByID

```
void StudyModule::searchByID(int
    searchID) {
    int left = 0, right = count - 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (deck[mid].id == searchID) {
            cout << "Found: " << deck[mid]
                .topic;
            return;
        }
        if (deck[mid].id < searchID)
            left = mid + 1;
        else right = mid - 1;
    }
}
```

Data Structures (Question.h)

```
• • • Question.h

struct Question {
    int id;
    string topic;
    string prompt;
    string answer;

    // Core metric for spaced
    // repetition
    int streak;
};
```

The Foundation of the Engine

A custom struct is utilized to model individual flashcards, combining essential text data with necessary performance metrics.

- **Unique ID:** Enables rapid $O(\log n)$ binary search lookups.
- **Streak Tracking:** The crucial integer variable that dictates when a question should reappear in the spaced repetition loop.

Object-Oriented Design (StudyModule.h)

Inheritance & Polymorphism

The system leverages advanced OOP concepts to keep the codebase modular, reusable, and secure.

- **Encapsulation:** Protected memory allocation (*deck pointer) prevents external tampering.
- **Polymorphism:** displayHint() is a pure virtual function, forcing derived classes to provide specific hint logic.

```
StudyModule.h

class StudyModule {
protected:
    Question *deck; // Dynamic array
                    pointer
    int count, capacity;
public:
    // ... standard study & I/O
    methods ...
    virtual void displayHint() = 0; //
    Pure virtual
};

class AlgorithmQuiz : public
    StudyModule {
public:
    void displayHint() override;
};
```

Raw Datasets (CSV)

The engine dynamically ingests external databases at runtime. The **Streak** column is actively read into memory, modified during study, and overwritten on exit to save state.

algorithms_questions.csv

```
ID,Topic,Prompt,Answer,Streak
201,Data Structures,What is the
    time complexity...,0(log n),2
202,Data Structures,What is the
    primary difference...,A stack
    is LIFO...,2
203,Data Structures,What is a
    memory leak in C++?,When a
    program...,1
```

ml_systems_questions.csv

```
ID,Topic,Prompt,Answer,Streak
401,System Design,What is the
    purpose of a Load Balancer?,To
    distribute...,2
402,System Design,What is the
    difference between vertical
    ...,Vertical...,0
501,Machine Learning,What is
    overfitting in Machine
    Learning?,When a model...,1
```

Execution: Study Loop

This simulation perfectly demonstrates the core logic in action.

- **Interactive Prompts:** The user is challenged with a specific question.
- **Polymorphic Hints:** The user requested a hint by pressing 'h', successfully calling the overridden method.
- **Self-Grading:** After the answer is revealed, the user grades themselves to update their internal streak variable.

```
Terminal

0 [ ] > ./StudyApp
Initializing system and loading
    datasets...
System ready.
=====

    CS \& AI SPACED REPETITION ENGINE
=====

1. Start Algorithm Quiz
2. Start Machine Learning Quiz
Choose an option: 1

--- Starting Study Session ---
Topic: Data Structures
Q: What is a memory leak in C++?
Press Enter for answer, 'h' for hint
    : h
[HINT]: Think about Big O Time
    Complexity...
```

Execution: $O(\log n)$ Search

Demonstrating the highly efficient Binary Search functionality across multiple datasets.

```
Terminal

=====

  CS \& AI SPACED REPETITION ENGINE
=====

Choose an option: 3

Enter Question ID to search: 203

Searching Algorithm Database...
--- Question Found ---
Topic: Data Structures
Prompt: What is a memory leak in C
      ++?
Answer: When a program dynamically
      ...
Current Streak: 1

Searching ML Database...
Question ID not found.
```

- **Instant Retrieval:** The user searched for ID 203.
- **Multi-DB Scanning:** The system natively checks the Algorithm Database first, successfully printing the specific prompt.
- **Graceful Fails:** It subsequently checks the ML Database and cleanly reports the ID is not found.

Execution: **Analytics**

Upon exiting the study loop (by pressing 'q'), the program calculates and prints session statistics.

- **Real-Time Tracking:** Variables like sessionTotalAnswered and sessionCorrect are tracked dynamically.
- **Dynamic Accuracy:** This shows the math working accurately with fractions (e.g., 3 correct out of 5 equals 60% accuracy).

```
Terminal

Session Ended.

=====
                SESSION STATISTICS
=====
Questions Answered: 5
Correct:             3
Incorrect:           2
Accuracy:            60%
=====

=====

                CS \& AI SPACED REPETITION ENGINE
=====

Choose an option: _
```

Execution: Safe Exit & Storage

Ensuring user data is never lost.

```
Terminal

=====

CS \& AI SPACED REPETITION ENGINE
=====

Choose an option: 4

Saving progress to databases...
Progress successfully saved to
    algorithms.csv.
Progress successfully saved to
    ml_systems.csv.
Exiting program. Goodbye!
0 █ > _
```

- **Menu Option 4:** Choosing to exit the program triggers the cleanup and storage sequence.
- **Overwriting Source:** The `saveToFile()` method successfully opens and overwrites the active CSV files with updated streak variables.
- **Graceful Termination:** The C++ program successfully reaches `return 0`;